Overflow, দশমিকসহ printf, escape অক্ষর

পাঠ 5 — 1IOT, SST-ICT-01 — char-এর সীমা, %f, \n \t \a

22/09/2026

1 দ্রুত পুনরালোচনা

এখন পর্যন্ত দেখা তিন ধরনের ভেরিয়েবল (**8 বিটের** `char`, 16 বিটের `int`, দশমিকসহ `float`) এবং আগে শেখা **গাণিতিক অপারেটর**, **মান নির্ধারণ** ও **তুলনার** অপারেটরের পুনরালোচনা।

2 Overflow সমস্যা: ডেটা টাইপ যখন খুব ছোট

উদাহরণ

বাড়ির কাজ: `prezzo` (দাম) আর `quantità` (পরিমাণ) নামে দুটি **ভেরিয়েবল** তৈরি করো, তাদের গুণফল `totale`-তে (মোট) হিসাব করো, ফলাফল ছাপাও।

```
char prezzo, quantita;
prezzo = 13;
quantita = 22;

char totale;
totale = prezzo * quantita;

printf("%d", totale);
```

13 আর 22 ছোট সংখ্যা, তাই `char` টাইপ তাদের রাখার জন্য যথেষ্ট মনে হয়।

সতর্কতা

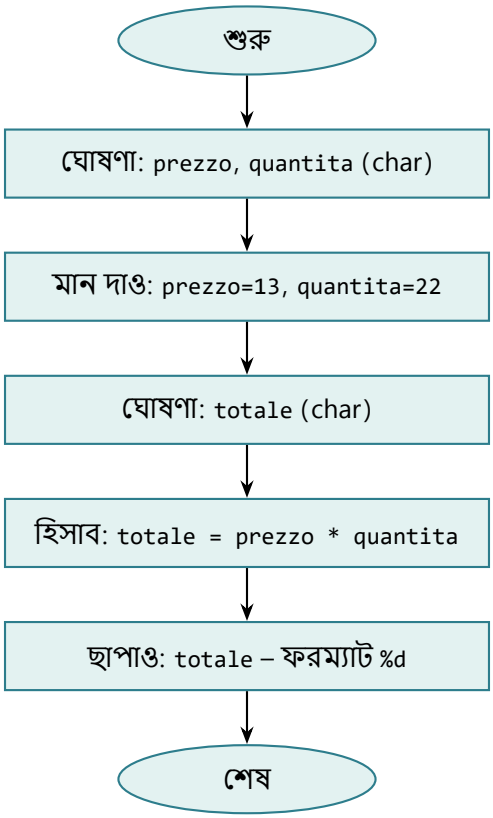
সমস্যা: ফলাফল হিসেবে 286 (13×22) না পেয়ে প্রোগ্রাম ছাপাত **30**। যন্ত্র কোনো **ভুল** জানাত না: প্রোগ্রাম "চলত", কিন্তু ফলাফল ভুল ছিল।

সংজ্ঞা

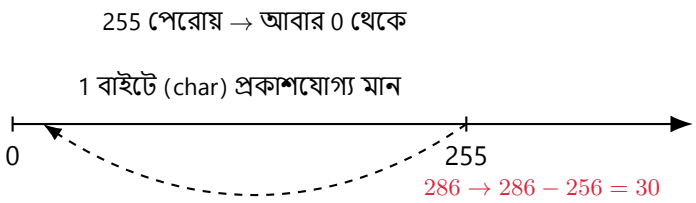
`char` টাইপ **8 বিটের** এবং শুধু 0 থেকে **255** পর্যন্ত মান রাখতে পারে। কোনো হিসাব 255 পেরোলে গণনা **আবার শূন্য থেকে শুরু হয়**: এই ঘটনাকে বলে **overflow**।

উদাহরণ

$286 - 256 = 30$: মানটি char-এর সর্বোচ্চ সীমা (256টি সম্ভাব্য মান, 0 থেকে 255) পেরিয়ে "ঘুরে" **আবার শূন্য থেকে শুরু** করে, পৌঁছায় 30-এ। তাই 286-এর বদলে 30 ছাপা হতো।



ছবি/নকশা 1: `char totale` সহ কোডের ফ্লোচার্ট: নির্দেশের একটি সরল ধারা, কোনো পছন্দ নেই – তবু overflow-এর কারণে শেষ ফলাফল ভুল।



ছবি/নকশা 2: char টাইপের overflow: 255 পেরোনো মান ভুল জানায় না, বরং "ঘুরে" আবার শূন্য থেকে শুরু করে।

সংজ্ঞা

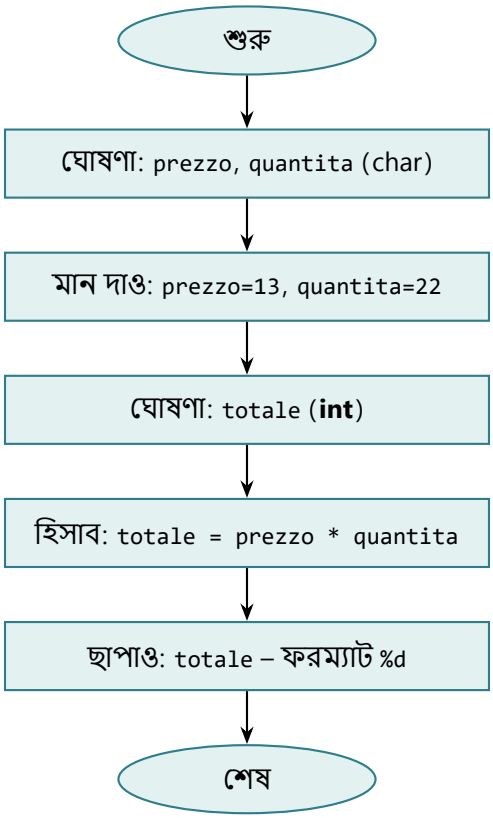
সংশোধন: যে **ভেরিয়েবল** ফলাফল পায়, তার টাইপ যথেষ্ট **বড়** হতে হবে। এখানে `totale`-কে `int` হিসেবে ঘোষণা করলেই হয়:

```
char prezzo, quantita;
prezzo = 13;
quantita = 22;

int totale;
totale = prezzo * quantita;

printf("%d", totale);
```

`prezzo` আর `quantita`-কেও **`int`** হিসেবে ঘোষণা করা যায়: হিসাবের জন্য দরকার নেই, তবে কোড আরও একরকম দেখায়।



ছবি/নকশা 3: সঠিক সংস্করণের ফ্লোচার্ট: আগের মতোই ধারা, কিন্তু `totale` এখন `int`, তাই হিসাবে আর overflow হয় না।

টীকা

যে `totale` **ভেরিয়েবল** ছাড়া সরাসরি `printf`-এর ভেতরে `prezzo * quantita` হিসাব করে, সেও সঠিক ফলাফল পায় – কিন্তু এটি ভালো অভ্যাস নয়: এভাবে প্রতিবার ছাপানোর সময় হিসাবটি **আবার করা হয়**, অকারণে গণনাশক্তি খরচ হয়। একবার একটি **ভেরিয়েবলে** হিসাব করে তারপর সেটি ছাপানো ভালো।

সতর্কতা

শিক্ষা: ব্যবহৃত **ভেরিয়েবলের ডেটা টাইপ** আর আকার সবসময় খেয়াল করো। অভ্যাসবশত সবসময় `char` বা সবসময় `int` লেখা সহজ, আর ভুলে যাওয়া যায় যে হিসাবের ফলাফল বেছে নেওয়া টাইপের ধারণক্ষমতা পেরোতে পারে কিনা।

3 দশমিকসহ সংখ্যা: %f নির্দেশক

উদাহরণ

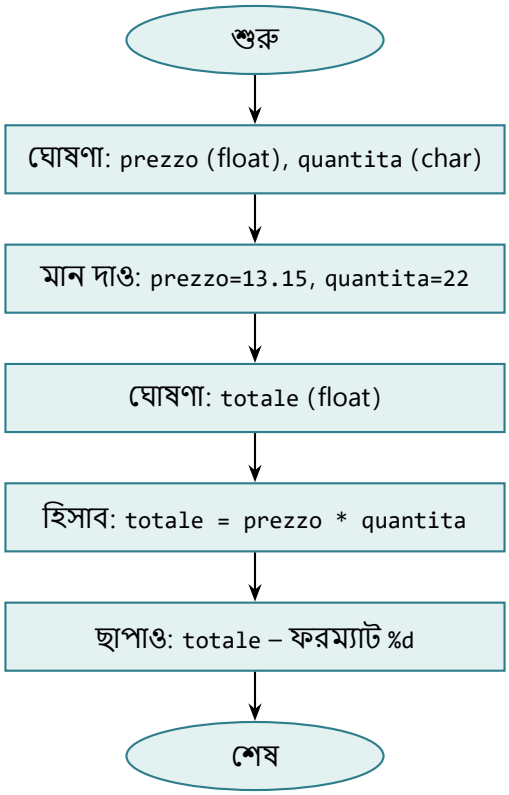
অনুশীলনের একটি রূপ: দাম যদি 13,15 হয় (**দশমিকসহ**), কী বদলায়?
প্রথম পর্যবেক্ষণ: `int` হিসেবে ঘোষিত **ভেরিয়েবল** `quantita` 4 **বাইট** জায়গা নেয়, যদিও 22-এর মতো মানের জন্য ছোট টাইপই যথেষ্ট – বড় সংখ্যা দরকার না হলে এটি **মেমোরির** অপচয়। কোন টাইপ বাছবে তা সবসময় **প্রসঙ্গের** ওপর নির্ভর করে: পূর্ণ পরিমাণ (মানুষ, টুকরো) **পূর্ণসংখ্যাই** থাকে; দশমিক লাগে এমন পরিমাণের (যেমন 1 কিলো 22 গ্রাম ময়দা) জন্য **`float`** লাগে।
সঠিক সংশোধন হলো শুধু `prezzo`-তে **`float`** দেওয়া:

```
float prezzo;
char quantita;

prezzo = 13.15;
quantita = 22;

float totale;
totale = prezzo * quantita;

printf("%d", totale);
```



ছবি/নকশা 4: দশমিকসহ `prezzo`-র ফ্লোচার্ট: ধারা একই, কিন্তু `float` ছাপাতে `%d` অর্থহীন ফলাফল দেয়।

সতর্কতা

`%d` দিয়ে ছাপানো ফলাফলের কোনো অর্থ নেই, কারণ `%d` `printf`-কে বলে "এখানে একটি **পূর্ণসংখ্যা** আসছে": তাকে **দশমিকসহ** মান দিলে পর্দায় যা দেখা যায় তার সাথে আসল হিসাবের কোনো সম্পর্ক থাকে না (ক্লাসের উদাহরণে `289,3`-এর বদলে `289` ছাপা হয়েছিল, কারণ ব্যাখ্যার সময় দশমিক অংশ হারিয়ে যাচ্ছিল)।

সংজ্ঞা

দশমিকসহ সংখ্যা ঠিকভাবে ছাপাতে `%d`-এর বদলে `%f` **নির্দেশক** ব্যবহার হয়:

```
printf("%f", totale);
```

3.1 দশমিকের পরের অঙ্কের সংখ্যা নিয়ন্ত্রণ

উদাহরণ

শুধু `%f` দিলে দশমিকের পরে অনেক অঙ্ক ছাপা হয় (প্রচলিত ফরম্যাটে ছয়টি)। কতগুলো অঙ্ক দেখাবে তা ঠিক করতে `%` আর `f`-এর মাঝে `0.` **ধারাটি** আর তারপর কাঙ্ক্ষিত অঙ্কের সংখ্যা লেখা হয়:

```
printf("%0.1f", totale);    // দশমিকের পরে একটি অঙ্ক
printf("%0.2f", totale);    // দশমিকের পরে দুটি অঙ্ক
printf("%0.3f", totale);    // দশমিকের পরে তিনটি অঙ্ক
printf("%0.10f", totale);   // দশমিকের পরে দশটি অঙ্ক
```

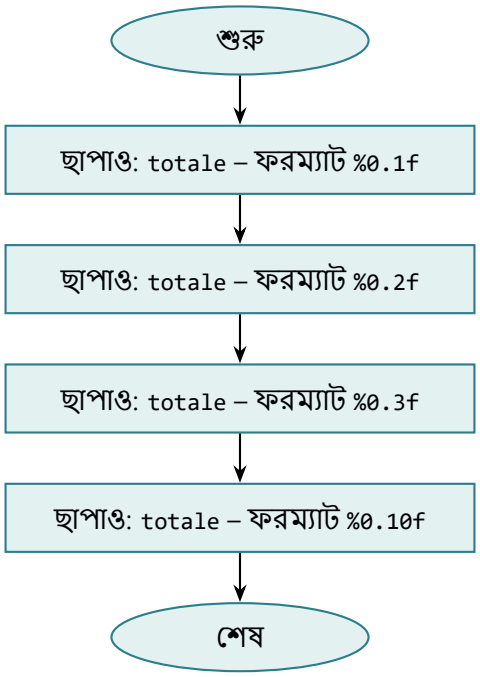
এই লেখার নিয়ম শুধু `%f`-এর সাথে কাজ করে: `%d`-এর সাথে এর মানে নেই, কারণ পূর্ণসংখ্যার দেখানোর মতো **দশমিক** **অঙ্ক** নেই।

টীকা

`%f` দিয়ে ছাপানো মান **বেছে** **নেওয়া** দশমিক অঙ্কে **গোল** **করা** হয়, কাটা হয় না। **পূর্ণসংখ্যায়** রূপান্তরের আচরণ আলাদা: সেখানে গোল না করে দশমিক অংশ কেটে ফেলা হয়।

উদাহরণ

কতগুলো **দশমিক** **অঙ্ক** ব্যবহার করবে তা প্রসঙ্গের ওপর নির্ভর করে: একটি পণ্যের দামের জন্য সাধারণত দুটি অঙ্কই যথেষ্ট (সেন্ট)। ইলেকট্রনিক উপাদানের পরিমাপে দশমিকের পরে 6, 7 বা 8 অঙ্ক লাগতে পারে, তাই বেশি অঙ্কের ফরম্যাট দরকার (যেমন `%0.8f` বা তার বেশি)।



ছবি/নকশা 5: ফ্লোচার্ট: একই মানের চারটি ধারাবাহিক ছাপা, প্রতিটিতে দশমিকের পরের অঙ্কের সংখ্যা আলাদা।

টীকা

ক্লাসের ঘটনা: $13{,}15 \times 22$ উদাহরণটি (প্রত্যাশিত ফলাফল: $289{,}3$) সরাসরি চালিয়ে দেখার সময় প্রোগ্রাম অনেক বেশি অঙ্কের একটি অস্বাভাবিক মান ছাপায়, যা আপাতদৃষ্টিতে `quantita` তখনও `char` থাকার সাথে সম্পর্কিত ছিল। শিক্ষক `quantita`-কে প্রথমে `int`, পরে `float` করে দেখেন, কিন্তু অস্বাভাবিকতা যায়নি, এবং তিনি স্বীকার করেন যে তখন সঠিক কারণ জানেন না; তিনি অনুমান করেন কম্পাইলের জন্য ব্যবহৃত অনলাইন ভার্চুয়াল মেশিনের কোনো বিশেষ আচরণ (সম্ভবত ভাসমান দশমিক প্রকাশের সমস্যা, যা তখন স্পষ্ট হয়নি)। এই ঘটনা `%f` ফরম্যাটের ব্যাখ্যা চালিয়ে যেতে বাধা দেয়নি।

4 Escape অক্ষর

সংজ্ঞা

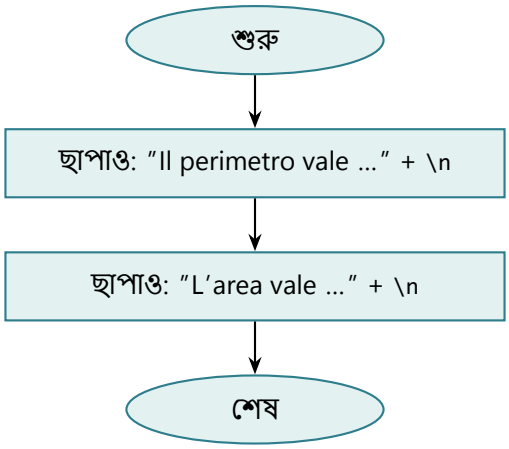
Escape অক্ষর (**carattere di escape**) হলো একটি বিশেষ ধারা, যা সবসময় **backslash** (\) প্রতীক দিয়ে শুরু হয়; এটি আক্ষরিক লেখা হিসেবে ছাপা হয় না, বরং **আউটপুটের** (**output**) বিন্যাসে একটি প্রভাব ফেলে।

4.1 \n — নতুন লাইন

উদাহরণ

উদ্‌ধৃতি চিহ্নের ভেতরের কোনো **স্ট্রিংয়ে** (**stringa**, লেখার ধারা) `\n` থাকলে, সেই জায়গায় আউটপুট **নতুন লাইনে যায়**:

```
printf("Il perimetro vale %0.2f\n", perimetro);
printf("L'area vale %0.2f\n", area);
```



ছবি/নকশা 6: ফ্লোচার্ট: দুটি ধারাবাহিক ছাপা, প্রতিটির পরে একটি `\n` যা আউটপুটকে নতুন লাইনে পাঠায়।

সতর্কতা

C কোড আলাদা লাইনে লেখা (যেমন দুটি আলাদা `printf`, প্রতি লাইনে একটি) মানেই আউটপুট নতুন লাইনে যাবে এমন **নয়**। **সোর্স কোডে** নতুন লাইনের **আউটপুটে** কোনো প্রভাব নেই: স্ট্রিংয়ের ভেতরে `\n` না থাকলে আউটপুট একই লাইনে থাকে, যদিও যে কোড তা তৈরি করে সে আলাদা লাইনে আছে।

উদাহরণ

ক্লাসে দেখানো: `"oggi é una"` লিখে, এবং দ্বিতীয় একটি `printf`-এ `"bella giornata"` লিখে, দুই স্ট্রিংয়ের মাঝে `\n` বা ফাঁক ছাড়া, আউটপুট হয় `oggi é unabella giornata` – সব জোড়া লাগা, কোড যেভাবেই লাইনে ভাগ করা থাকুক। প্রথম **স্ট্রিংয়ের** শেষে একটি ফাঁক বা `\n` যোগ করলে ফলাফল আবার পড়ার মতো হয়।
পরপর একাধিক `\n` বসানো যায়: পরপর দুটি `\n` দুবার নতুন লাইনে যায়, অর্থাৎ মাঝে একটি খালি লাইন রাখে।

4.2 \t — ট্যাব

সংজ্ঞা

`\t` একটি **ট্যাব** (**tabulazione**) করে: নির্দিষ্ট সংখ্যক ফাঁক বসায় (সাধারণত এডিটরে ঠিক করা যায়, প্রায়ই মান অনুযায়ী 3 বা 4)।

টীকা

কয়েকবার স্পেসবার চাপার বদলে `\t` কেন? কারণ `\t` **মেমোরিতে** **একটিমাত্র** **বাইট** নেয়, আর আলাদাভাবে টাইপ করা 4টি ফাঁকে লাগে 4 বাইট। কয়েক লাইনের লেখায় পার্থক্য নগণ্য, কিন্তু খুব লম্বা লেখায় (অনেক পুনরাবৃত্ত ফাঁকসহ) **ট্যাব** ব্যবহার **মেমোরির** জায়গা অনেকটা কমাতে পারে।

4.3 \a — শব্দ সংকেত

সংজ্ঞা

\a আউটপুটে এলে একটি **শব্দ** **সংকেত** (beep) দেয়।

উদাহরণ

এই অক্ষরের একটি বাস্তব ব্যবহার: সুপারমার্কেটের বারকোড রিডারে স্ক্যানের সময় একটি beep জানায় যে পড়া সফল হয়েছে – রিডার কোনো শব্দ না করলে কর্মী **পর্দা** না দেখেই সঙ্গে সঙ্গে বোঝে কিছু কাজ করেনি।

5 নির্দেশিত অনুশীলন: বৃত্তের পরিধি ও ক্ষেত্রফল

উদাহরণ

সমস্যা (বোর্ড): ব্যাসার্ধ r দেওয়া আছে, বৃত্তের **পরিধি** (**perimetro**) ও ক্ষেত্রফল (**area**) হিসাব করে ফলাফল ছাপাও।

$$P = 2 \cdot \pi \cdot r$$
$$A = r \cdot r \cdot \pi$$

যেখানে $\pi = 3{,}141592$।

উদাহরণ

ক্লাসে একসাথে লেখা কোড:

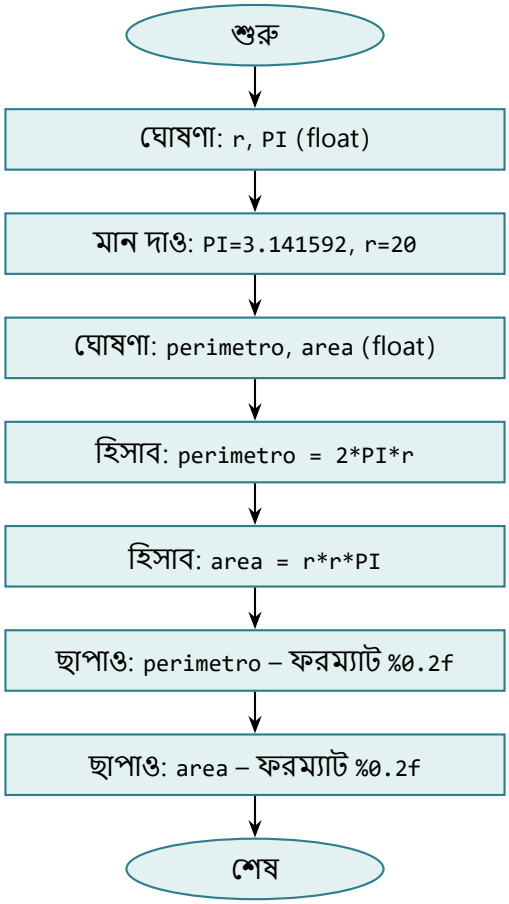
```
float r;
float PI;

PI = 3.141592;
r = 20;

float perimetro;
float area;

perimetro = 2 * PI * r;
area = r * r * PI;

printf("%0.2f", perimetro);
printf("%0.2f", area);
```



ছবি/নকশা 7: বৃত্তের পরিধি ও ক্ষেত্রফল হিসাবের ফ্লোচার্ট: ঘোষণা, মান নির্ধারণ, দুটি হিসাব এবং শেষের দুটি ছাপা, সব ধারাবাহিকভাবে।

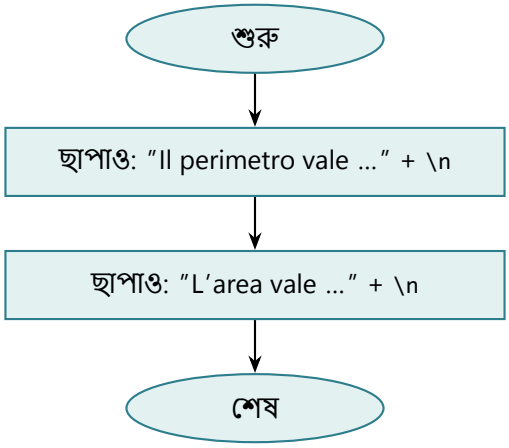
সতর্কতা

পড়ার সমস্যা: কোড চালালে **আউটপুট** হয় অঙ্ক আর বিন্দুর একটি গোলমেলে **ধারা** (দুটি ফলাফল একসাথে জোড়া, যেমন `125.661256.64`), কারণ দুই `printf`-এর মাঝে কোনো ফাঁক বা `\n` নেই। একটি ফাঁক দিলে পড়া যায় (`125.66 1256.64`), কিন্তু অর্থ বোঝা যায় না: পাঠক জানে না কোন সংখ্যাটি **পরিধি** আর কোনটি ক্ষেত্রফল।

সংজ্ঞা

সমাধান: প্রতিটি `%f`-এর আগে একটি বর্ণনামূলক লেখা যোগ করো, যাতে ছাপানো সংখ্যার **প্রসঙ্গ** বোঝা যায়:

```
printf("Il perimetro vale %0.2f\n", perimetro);
printf("L'area vale %0.2f\n", area);
```



ছবি/নকশা 8: প্রসঙ্গসহ সমাধানের ফ্লোচার্ট: দুটি ধারাবাহিক ছাপা, প্রতিটিতে সংখ্যাটি ব্যাখ্যা করা একটি লেখা।

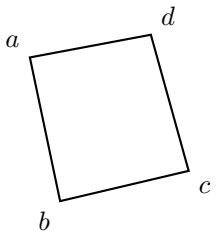
টীকা

আউটপুটে সংখ্যার **প্রসঙ্গ** দেওয়া কেন জরুরি? লেবেল ছাড়া একটি সংখ্যা পাঠককে কিছুই জানায় না, বিশেষ করে পাঠক যদি প্রোগ্রামের লেখক না হয়। প্রোগ্রাম শুধু `125,66` ছাপালে, শুধু লেখকই জানে এটি একটি **পরিধি**: অন্য কেউ (যেমন কোড না জেনে প্রোগ্রাম ব্যবহারকারী) তা বোঝার উপায় পায় না। তাই অন্যদের ব্যবহারের জন্য তৈরি প্রোগ্রামে প্রতিটি ছাপানো সংখ্যার সাথে তার অর্থ বোঝানো একটি লেখা থাকা উচিত।

6 অনুশীলন: 4 বাহুর বহুভুজের পরিসীমা

উদাহরণ

সমস্যা (বোর্ড): 4 বাহুর একটি বহুভুজের বাহু a, b, c, d (মান আগে থেকেই দেওয়া), একটি প্রোগ্রাম লেখো যা **পরিসীমা** (**perimetro**, বাহুগুলোর যোগফল) হিসাব করে ছাপায়।

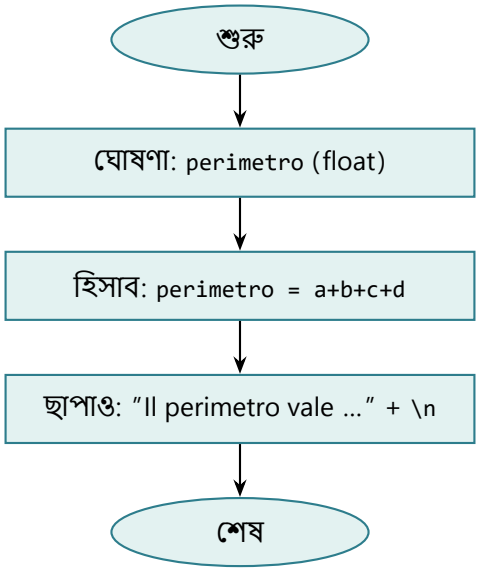


ছবি/নকশা 9: বোর্ডে আঁকা চতুর্ভুজ, চারটি বাহু a, b, c, d দিয়ে চিহ্নিত।

উদাহরণ

সমাধান, **আউটপুটের** প্রসঙ্গ নিয়ে যা দেখেছি তা প্রয়োগ করে:

```
float perimetro;
perimetro = a + b + c + d;
printf("Il perimetro vale %0.2f\n", perimetro);
```



ছবি/নকশা 10: চতুর্ভুজের পরিসীমা হিসাবের ফ্লোচার্ট: ঘোষণা, চার বাহুর যোগফল, প্রসঙ্গসহ ছাপা।

7 শেষ অনুশীলন: একটিমাত্র printf

উদাহরণ

সমস্যা: একটি প্রোগ্রাম লেখো যা তারকাচিহ্ন দিয়ে একটি **ভরা** **আয়তক্ষেত্র** ছাপায়, 5 অক্ষরের 3টি লাইন, **একটিমাত্র** `printf` ব্যবহার করে (একটির বেশি অনুমোদিত নয়):

```
*****
*****
*****
```

সতর্কতা

অনুশীলনের শর্ত: পুরো **আউটপুট** একটিমাত্র `printf` নির্দেশ থেকে আসতে হবে। ছাপানোর মতো সংখ্যা না থাকায় সমাধান হলো একটি একক লেখার **স্ট্রিং**, যাতে তারকাচিহ্ন আর এক লাইন থেকে পরের লাইনে যাওয়ার **escape** **অক্ষর** `\n` মেশানো:

```
printf("*****\n*****\n*****\n");
```

উদাহরণ

আরেকটি রূপ: একই আয়তক্ষেত্র ছাপাও, কিন্তু **ফাঁপা** (শুধু তারকাচিহ্নের সীমানা, মাঝখান খালি), সেই একটিমাত্র `printf` দিয়ে:

```
*****
*   *
*****
```

সমাধান: মাঝের লাইনের ফাঁকগুলো তারকাচিহ্নের মতোই সাধারণ অক্ষর, **স্ট্রিংয়ের** ভেতরে স্পষ্ট করে লিখতে হয়:

```
printf("*****\n*   *\n*****\n");
```

টীকা

এটি এই পাঠের সবকিছুর একটি সারসংক্ষেপ অনুশীলন: নতুন লাইনের জন্য escape অক্ষর `\n`, এবং একটিমাত্র `printf` নির্দেশ দিয়ে জটিল (বহু-লাইনের) **আউটপুট** তৈরি করা, ফরম্যাটের **স্ট্রিং** অক্ষরে অক্ষরে গড়ে।

৪ এই পাঠ থেকে যা মনে রাখতে হবে

- **Overflow**: খুব ছোট টাইপ (যেমন `char`, সর্বোচ্চ 255) **ভুল** জানায় না, বরং "ঘুরে" **আবার** **শূন্য** **থেকে** **শুরু** **করে**, সতর্কতা ছাড়াই ভুল ফলাফল দেয়।
- সবসময় **ভেরিয়েবলে** রাখতে হবে এমন সর্বোচ্চ মান দেখে **ডেটা** **টাইপ** বেছে নাও।
- `%d` **পূর্ণসংখ্যার** জন্য, `%f` **দশমিকসহ** সংখ্যার জন্য – এগুলো অদলবদল করা যায় না।
- `%0.Nf` ছাপানো **দশমিক** **অঙ্কের** সংখ্যা `N` নিয়ন্ত্রণ করে, মান গোল করে।
- **Escape** **অক্ষর** শুরু হয় `\` দিয়ে: `\n` **নতুন** **লাইনে** **যায়**, `\t` ট্যাব দেয়, `\a` beep দেয়।
- **সোর্স** **কোডে** নতুন লাইন মানে **আউটপুটে** নতুন লাইন নয়: সবসময় স্পষ্ট `\n` দরকার।
- ছাপানো সংখ্যার সাথে সবসময় তার অর্থ বোঝানো লেখা দিয়ে **প্রসঙ্গ** দিতে হবে।

উদাহরণ

পরের পাঠের জন্য করণীয়: একটিমাত্র `printf` দিয়ে তারকাচিহ্নের ফাঁপা **আয়তক্ষেত্রের** অনুশীলন শেষ করো, আর 4 বাহুর বহুভুজের **পরিসীমার** হিসাবও ইচ্ছামতো একটি সংখ্যার উদাহরণ দিয়ে যাচাই করো।

9 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **prezzo / quantità / totale**: দাম / পরিমাণ / মোট
- **a capo**: নতুন লাইনে
- **tabulazione**: ট্যাব
- **stringa**: স্ট্রিং (লেখার ধারা)
- **raggio**: ব্যাসার্ধ
- **perimetro / area**: পরিসীমা (বৃত্তে পরিধি) / ক্ষেত্রফল
- **rettangolo pieno / vuoto**: ভরা / ফাঁপা আয়তক্ষেত্র
- **cifre decimali**: দশমিকের পরের অঙ্ক
- **Il perimetro vale**: পরিসীমার মান (কোডে ছাপানো লেখা)
- **L’area vale**: ক্ষেত্রফলের মান (কোডে ছাপানো লেখা)